Solving Inverse Problems: Variational Regularization vs. CNNs

STATS 231B: Methods of Machine Learning

Eduard Bueno, Carson Newman, Keshav Singh, Justin Tran, Ethan Young

Department of Statistics and Data Science

June 1, 2026



Inverse Problems and Ill-Posedness

We observe corrupted indirect measurements

$$y = Hx + n.$$

- $x \in \mathbb{R}^d$: unknown clean image or signal.
- $y \in \mathbb{R}^m$: measured data.
- $H \in \mathbb{R}^{m \times d}$: forward operator, e.g. blur, subsampling, CT projection.
- $n \in \mathbb{R}^m$: noise or model error.

The problem is **ill-posed** when recovering x from y is unstable or non-unique.

For example, if H is non-invertible or ill-conditioned, small changes in y can cause large changes in the reconstruction.

Inverse Problems and Ill-Posedness

We observe corrupted indirect measurements

$$y = Hx + n.$$

- $x \in \mathbb{R}^d$: unknown clean image or signal.
- $y \in \mathbb{R}^m$: measured data.
- $H \in \mathbb{R}^{m \times d}$: forward operator, e.g. blur, subsampling, CT projection.
- $n \in \mathbb{R}^m$: noise or model error.

The problem is **ill-posed** when recovering x from y is unstable or non-unique.

For example, if H is non-invertible or ill-conditioned, small changes in y can cause large changes in the reconstruction.

Variational Regularization

A classical way to stabilize the inverse problem is to reconstruct x by solving

$$\widehat{x}(y) = \arg\min_x \{\mathcal{D}(Hx, y) + \alpha\mathcal{R}(x)\}.$$

- $\mathcal{D}(Hx, y)$: data fidelity term (loss function).
- $\mathcal{R}(x)$: regularizer; encodes prior structure such as smoothness or sparsity.
- $\alpha > 0$: regularization strength; balances measurement fit and prior structure.

Bayesian Correspondence

From a Bayesian viewpoint, the unknown image is modeled as a random variable

$$X \sim P_X.$$

Bayes' rule gives

$$\underbrace{p_{X|Y}(x \mid y)}_{\text{posterior}} \propto \underbrace{p_{Y|X}(y \mid x)}_{\text{likelihood}} \quad \underbrace{p_X(x)}_{\text{clean image prior}} .$$

The maximum a posteriori (MAP) estimator is

$$\widehat{x}_{\mathrm{MAP}} = \arg\min_x \left\{ -\log p_{Y|X}(y \mid x) - \log p_X(x) \right\}.$$

Thus, variational regularization corresponds to

$$p_{Y|X}(y \mid x) \propto e^{-\mathcal{D}(Hx,y)}, \qquad p_X(x) \propto e^{-\alpha \mathcal{R}(x)}.$$

Bayesian Correspondence

From a Bayesian viewpoint, the unknown image is modeled as a random variable

$$X \sim P_X.$$

Bayes' rule gives

$$\underbrace{p_{X|Y}(x \mid y)}_{\text{posterior}} \propto \underbrace{p_{Y|X}(y \mid x)}_{\text{likelihood}} \underbrace{p_X(x)}_{\text{clean image prior}}.$$

The maximum a posteriori (MAP) estimator is

$$\widehat{x}_{\mathrm{MAP}} = \arg\min_x \left\{ -\log p_{Y|X}(y \mid x) - \log p_X(x) \right\}.$$

Thus, variational regularization corresponds to

$$p_{Y|X}(y \mid x) \propto e^{-\mathcal{D}(Hx,y)}, \qquad p_X(x) \propto e^{-\alpha \mathcal{R}(x)}.$$

Convex Learned Regularizers: ICNNs

The main design choice is the regularizer

$$\mathcal{R}_\theta(x).$$

An **input convex neural network**[1] is a scalar-valued neural network whose output is convex as a function of its input x.

$$z_1 = g_0(W_0^{(x)}x + b_0),$$
$$z_{i+1} = g_i(W_i^{(z)}z_i + W_i^{(x)}x + b_i), \qquad i = 1, \dots, \ell - 1.$$

We set $\mathcal{R}_\theta(x) = z_\ell$. Convexity is enforced by requiring:

- g_i coordinatewise convex and nondecreasing (e.g. ReLU);
- $W_i^{(z)}$ coordinatewise nonnegative;
- $W_i^{(x)}$ unrestricted affine input connections.

[1] Amos, Xu, and Kolter, "Input Convex Neural Networks", 2017

Convex Learned Regularizers: ICNNs

The main design choice is the regularizer

$$\mathcal{R}_\theta(x).$$

An **input convex neural network**[1] is a scalar-valued neural network whose output is convex as a function of its input x.

$$\begin{aligned} z_1 &= g_0(W_0^{(x)}x + b_0), \\ z_{i+1} &= g_i(W_i^{(z)}z_i + W_i^{(x)}x + b_i), \qquad i = 1, \dots, \ell - 1. \end{aligned}$$

We set $\mathcal{R}_\theta(x) = z_\ell$. Convexity is enforced by requiring:

- g_i coordinatewise convex and nondecreasing (e.g. ReLU);
- $W_i^{(z)}$ coordinatewise nonnegative;
- $W_i^{(x)}$ unrestricted affine input connections.

[1] Amos, Xu, and Kolter, "Input Convex Neural Networks", 2017

Convex Learned Regularizers: ICNNs

The main design choice is the regularizer

$$\mathcal{R}_\theta(x).$$

An **input convex neural network**[1] is a scalar-valued neural network whose output is convex as a function of its input x.

$$\begin{aligned} z_1 &= g_0(W_0^{(x)}x + b_0), \\ z_{i+1} &= g_i(W_i^{(z)}z_i + W_i^{(x)}x + b_i), \qquad i = 1, \dots, \ell - 1. \end{aligned}$$

We set $\mathcal{R}_\theta(x) = z_\ell$. Convexity is enforced by requiring:

- g_i coordinatewise convex and nondecreasing (e.g. ReLU);
- $W_i^{(z)}$ coordinatewise nonnegative;
- $W_i^{(x)}$ unrestricted affine input connections.

[1] Amos, Xu, and Kolter, "Input Convex Neural Networks", 2017

Motivation

If ICNNs guarantee convexity, why not stop there?

The Trade-off:

- **ICNNs (Convex):** Provide strong mathematical convergence guarantees, but the structural restrictions limit expressiveness. This leads to blurry or suboptimal image reconstructions.
- **Standard CNNs (Non-convex):** Highly expressive and capture fine details, but introduce massive non-convexity. It loses guarantee of converging and can cause unstable training or hallucinated artifacts.

The Solution: Difference-of-Convex (DC) Functions[2].

- By using a broader class of structured non-convex functions, we can achieve state-of-the-art empirical performance (high expressivity) while retaining the mathematical structure needed for rigorous convergence guarantees.

[2] Zhang and Leong, "Learning Difference-of-Convex Regularizers for Inverse Problems: A Flexible Framework with Theoretical Guarantees", 2025

Motivation

If ICNNs guarantee convexity, why not stop there?

The Trade-off:

- **ICNNs (Convex):** Provide strong mathematical convergence guarantees, but the structural restrictions limit expressiveness. This leads to blurry or suboptimal image reconstructions.
- **Standard CNNs (Non-convex):** Highly expressive and capture fine details, but introduce massive non-convexity. It loses guarantee of converging and can cause unstable training or hallucinated artifacts.

The Solution: Difference-of-Convex (DC) Functions[2].

- By using a broader class of structured non-convex functions, we can achieve state-of-the-art empirical performance (high expressivity) while retaining the mathematical structure needed for rigorous convergence guarantees.

[2] Zhang and Leong, "Learning Difference-of-Convex Regularizers for Inverse Problems: A Flexible Framework with Theoretical Guarantees", 2025

Motivation

If ICNNs guarantee convexity, why not stop there?

The Trade-off:

- **ICNNs (Convex):** Provide strong mathematical convergence guarantees, but the structural restrictions limit expressiveness. This leads to blurry or suboptimal image reconstructions.
- **Standard CNNs (Non-convex):** Highly expressive and capture fine details, but introduce massive non-convexity. It loses guarantee of converging and can cause unstable training or hallucinated artifacts.

The Solution: Difference-of-Convex (DC) Functions[2].

- By using a broader class of structured non-convex functions, we can achieve state-of-the-art empirical performance (high expressivity) while retaining the mathematical structure needed for rigorous convergence guarantees.

[2] Zhang and Leong, "Learning Difference-of-Convex Regularizers for Inverse Problems: A Flexible Framework with Theoretical Guarantees", 2025

Input Difference-of-Convex Neural Networks (IDCNNs)

We parameterize our new regularizer as the difference of two convex networks:

$$\mathcal{R}_\theta(x) = \mathcal{R}_{\theta_1}(x) - \mathcal{R}_{\theta_2}(x)$$

- Both $\mathcal{R}_{\theta_1}$ and $\mathcal{R}_{\theta_2}$ are built using the strict ICNN architecture.
- While the individual components are convex, their difference $\mathcal{R}_\theta(x)$ is highly **non-convex**.
- This DC structure is strictly more expressive than weakly convex regularizers, allowing for better fitting to ground-truth data manifolds.

Proposition 1 (ZL25 Prop. 3.1)

If an optimal regularizer F is the difference of two Lipschitz convex functions over a compact domain K, then for any $\epsilon > 0$, there exists an IDCNN such that:

$$\sup_{x \in K} |F(x) - (\mathcal{R}_{\theta_1}(x) - \mathcal{R}_{\theta_2}(x))| \leq \epsilon$$

Setup and Theoretical Assumptions

To leverage this DC structure in our inverse problem, our overall minimization objective becomes:

$$F(x) := \underbrace{\frac{1}{2}\|Ax - y\|_2^2 + \mathcal{R}_{\theta_1}(x)}_{\text{Strictly Convex } (G(x))} - \underbrace{\mathcal{R}_{\theta_2}(x)}_{\text{Concave } (-H(x))}$$

For optimization guarantees to hold, we assume:

1. **Lower Boundedness:** The overall objective function $F(x)$ must be bounded from below (so it doesn't drift to $-\infty$).
2. **DC Structure:** The objective must be strictly separable into a convex component and a concave component.
3. **Smoothness:** The gradient of the convex component $\mathcal{R}_{\theta_1}$ is assumed to be Lipschitz continuous (L_1-smooth), which is standard for ensuring step sizes converge.

Setup and Theoretical Assumptions

To leverage this DC structure in our inverse problem, our overall minimization objective becomes:

$$F(x) := \underbrace{\frac{1}{2}\|Ax - y\|_2^2 + \mathcal{R}_{\theta_1}(x)}_{\text{Strictly Convex } (G(x))} - \underbrace{\mathcal{R}_{\theta_2}(x)}_{\text{Concave } (-H(x))}$$

For optimization guarantees to hold, we assume:

1. **Lower Boundedness:** The overall objective function $F(x)$ must be bounded from below (so it doesn't drift to $-\infty$).
2. **DC Structure:** The objective must be strictly separable into a convex component and a concave component.
3. **Smoothness:** The gradient of the convex component $\mathcal{R}_{\theta_1}$ is assumed to be Lipschitz continuous (L_1-smooth), which is standard for ensuring step sizes converge.

Optimization: Difference-of-Convex Algorithm (DCA)

Because $F(x)$ is non-convex, standard gradient descent might get trapped or behave unpredictably. Instead, we use the **Difference-of-Convex Algorithm (DCA)**.

Core Idea (Majorization-Minimization):
We tackle the difficult concave part $(-\mathcal{R}_{\theta_2})$ by linearizing it using its subgradient.

At iteration t with current estimate x_t:

- **Step 1:** Compute the subgradient of the second network: $g_t \in \partial \mathcal{R}_{\theta_2}(x_t)$
- **Step 2:** Form a strictly convex surrogate objective $q(x; x_t)$ using the first-order Taylor expansion:

$$q(x; x_t) := \frac{1}{2}\|Ax - y\|_2^2 + \mathcal{R}_{\theta_1}(x) - \mathcal{R}_{\theta_2}(x_t) - \langle g_t, x - x_t \rangle$$

Optimization: Difference-of-Convex Algorithm (DCA)

Because $F(x)$ is non-convex, standard gradient descent might get trapped or behave unpredictably. Instead, we use the **Difference-of-Convex Algorithm (DCA)**.

Core Idea (Majorization-Minimization):
We tackle the difficult concave part $(-\mathcal{R}_{\theta_2})$ by linearizing it using its subgradient.

At iteration t with current estimate x_t:

- **Step 1:** Compute the subgradient of the second network: $g_t \in \partial \mathcal{R}_{\theta_2}(x_t)$
- **Step 2:** Form a strictly convex surrogate objective $q(x; x_t)$ using the first-order Taylor expansion:

$$q(x; x_t) := \frac{1}{2}\|Ax - y\|_2^2 + \mathcal{R}_{\theta_1}(x) - \mathcal{R}_{\theta_2}(x_t) - \langle g_t, x - x_t \rangle$$

DCA Convergence Guarantees

- **Step 3:** Solve the convex sub-problem to find the next iterate:

$$x_{t+1} \in \arg\min_{x \in \mathbb{R}^d} q(x; x_t)$$

Why does this work?

Because the concave term is linearized, the surrogate function $q(x; x_t)$ acts as a strict upper bound to the true objective $F(x)$ at x_t.

Solving this sub-problem at each step strictly decreases the overall objective. The sequence of function values $\{F(x_t)\}$ is monotonically decreasing, guaranteeing convergence to a critical stationary point, despite the overall deep neural network being non-convex.

DCA Convergence Guarantees

- **Step 3:** Solve the convex sub-problem to find the next iterate:

$$x_{t+1} \in \arg\min_{x \in \mathbb{R}^d} q(x; x_t)$$

Why does this work?

Because the concave term is linearized, the surrogate function $q(x; x_t)$ acts as a strict upper bound to the true objective $F(x)$ at x_t.

Solving this sub-problem at each step strictly decreases the overall objective. The sequence of function values $\{F(x_t)\}$ is monotonically decreasing, guaranteeing convergence to a critical stationary point, despite the overall deep neural network being non-convex.

Motivation

Recall the goal of computing $\hat{x}$ as a solution of the variational problem

$$\hat{x} = \arg\min_{x \in \mathbb{R}^d} \frac{1}{2}\|Hx - y\|_2^2 + \mathcal{R}(x).$$

It is known that a well-chosen non-convex $\mathcal{R}$ can lead to improved performance (although finding the global minimum becomes generally intractable).

The Convex Non-Convex Framework

One remedy is the **convex non-convex** (CNC) framework, where the global objective is modified to be

$$\mathcal{J}(x) = \frac{1}{2}\|Hx - y\|_2^2 + \mathcal{R}_{\text{CNC}}(x).$$

Various strategies to design a weakly convex regularizer include:

- Replacing the $\|\cdot\|_1$ by a non-convex penalty function.
- Using the generalized Moreau envelope of $\mathcal{R}_{\text{CNC}}$.
- Learning the proximal operator of $\mathcal{R}_{\text{CNC}}$.

The Convex Non-Convex Framework

One remedy is the **convex non-convex** (CNC) framework, where the global objective is modified to be

$$\mathcal{J}(x) = \frac{1}{2}\|Hx - y\|_2^2 + \mathcal{R}_{\text{CNC}}(x).$$

Various strategies to design a weakly convex regularizer include:

- Replacing the $\|\cdot\|_1$ by a non-convex penalty function.
- Using the generalized Moreau envelope of $\mathcal{R}_{\text{CNC}}$.
- Learning the proximal operator of $\mathcal{R}_{\text{CNC}}$.

Weakly Convex Regularizers

For the task of image denoising, it was recently proposed to use weakly-convex regularizers[3]. More precisely, $\mathcal{R}$ being ρ-weakly convex means that it satisfies

$$H_{\mathcal{R}}(x) \succeq -\rho I.$$

For the overall objective to be convex, we require that $\rho = 1$. This ensures that the proximal operator for $\mathcal{R}$ is well-defined and that the denoiser is Lipschitz continuous.

[3] Goujon, Neumayer, and Unser, "Learning Weakly Convex Regularizers for Convergent Image-Reconstruction Algorithms", 2024

Weakly Convex Regularizers

For the task of image denoising, it was recently proposed to use weakly-convex regularizers[3]. More precisely, $\mathcal{R}$ being ρ-weakly convex means that it satisfies

$$H_{\mathcal{R}}(x) \succeq -\rho I.$$

For the overall objective to be convex, we require that $\rho = 1$. This ensures that the proximal operator for $\mathcal{R}$ is well-defined and that the denoiser is Lipschitz continuous.

[3] Goujon, Neumayer, and Unser, "Learning Weakly Convex Regularizers for Convergent Image-Reconstruction Algorithms", 2024

Weakly Convex Regularizers

For the task of image denoising, it was recently proposed to use weakly-convex regularizers[3]. More precisely, $\mathcal{R}$ being ρ-weakly convex means that it satisfies

$$H_{\mathcal{R}}(x) \succeq -\rho I.$$

For the overall objective to be convex, we require that $\rho = 1$. This ensures that the proximal operator for $\mathcal{R}$ is well-defined and that the denoiser is Lipschitz continuous.

[3] Goujon, Neumayer, and Unser, "Learning Weakly Convex Regularizers for Convergent Image-Reconstruction Algorithms", 2024

Weakly Convex Regularizers

For the task of image denoising, it was recently proposed to use weakly-convex regularizers[3]. More precisely, $\mathcal{R}$ being ρ-weakly convex means that it satisfies

$$H_{\mathcal{R}}(x) \succeq -\rho I.$$

For the overall objective to be convex, we require that $\rho = 1$. This ensures that the proximal operator for $\mathcal{R}$ is well-defined and that the denoiser is Lipschitz continuous.

[3] Goujon, Neumayer, and Unser, "Learning Weakly Convex Regularizers for Convergent Image-Reconstruction Algorithms", 2024

WCRR-NN

The weakly-convex regularizer $\mathcal{R}$ is

$$\mathcal{R} : x[\cdot] \mapsto \sum_{i=1}^{N_C} \sum_{\mathbf{k} \in \mathbb{R}^2} \psi_i((h_i * x)[\mathbf{k}]).$$

Equivalently,

$$\mathcal{R} : \mathbf{x} \mapsto \sum_{j=1}^{d \times N_C} \psi_j(w_j^\top \mathbf{x}).$$

The proximal denoiser is

$$D_\theta(\mathbf{y}) = \arg\min_{\mathbf{x} \in \mathbb{R}^d} \frac{1}{2}\|\mathbf{x} - \mathbf{y}\|_2^2 + \mathcal{R}_\theta(\mathbf{x}).$$

WCRR-NN

The weakly-convex regularizer $\mathcal{R}$ is

$$\mathcal{R} : x[\cdot] \mapsto \sum_{i=1}^{N_C} \sum_{\mathbf{k} \in \mathbb{R}^2} \psi_i((h_i * x)[\mathbf{k}]).$$

Equivalently,

$$\mathcal{R} : \mathbf{x} \mapsto \sum_{j=1}^{d \times N_C} \psi_j(w_j^\top \mathbf{x}).$$

The proximal denoiser is

$$D_\theta(\mathbf{y}) = \arg\min_{\mathbf{x} \in \mathbb{R}^d} \frac{1}{2}\|\mathbf{x} - \mathbf{y}\|_2^2 + \mathcal{R}_\theta(\mathbf{x}).$$

WCRR-NN

The weakly-convex regularizer $\mathcal{R}$ is

$$\mathcal{R} : x[\cdot] \mapsto \sum_{i=1}^{N_C} \sum_{\mathbf{k} \in \mathbb{R}^2} \psi_i((h_i * x)[\mathbf{k}]).$$

Equivalently,

$$\mathcal{R} : \mathbf{x} \mapsto \sum_{j=1}^{d \times N_C} \psi_j(w_j^\top \mathbf{x}).$$

The proximal denoiser is

$$D_\theta(\mathbf{y}) = \arg\min_{\mathbf{x} \in \mathbb{R}^d} \frac{1}{2}\|\mathbf{x} - \mathbf{y}\|_2^2 + \mathcal{R}_\theta(\mathbf{x}).$$

WCRR-NN

We decompose the activation $\varphi = \psi'$ as the difference of two splines

$$\varphi = \mu\varphi_+ - \varphi_-,$$

and enforce the convolutional filters W to have unit spectral norm. This ensures the following properties:

- $\mathcal{R}$ will be 1-weakly convex.
- The Lipschitz constant of $\nabla\mathcal{R}$ is bounded.
- ρ is controlled by the spline profiles.

WCRR-NN

We decompose the activation $\varphi = \psi'$ as the difference of two splines

$$\varphi = \mu\varphi_+ - \varphi_-,$$

and enforce the convolutional filters W to have unit spectral norm. This ensures the following properties:

- $\mathcal{R}$ will be 1-weakly convex.
- The Lipschitz constant of $\nabla\mathcal{R}$ is bounded.
- ρ is controlled by the spline profiles.

Training

For the multi-noise setting, we learn the proximal denoiser $D_\theta(\mathbf{y})$ as follows:

1. Compute the spectrum of W using the discrete Fourier transform.
2. Compute the Jacobian of the denoiser via implicit differentiation.
3. Solve the training problem with stochastic Adam.

An End-to-End Blackbox Solution: CNNs

Deep Learning training advances continue to push the performance of Neural Networks to unmatched levels, so Zhang et al.[4] propose using CNNs to solve a special case of inverse problems: **Image Denoising**

$$y = x + v$$

We observe a noisy image y and we want to recover x ie remove the noise v from y. Instead of using a structured regularizer to enforce a prior, we could just use a CNN to try and recover the image directly.

[4] Zhang et al., "Beyond a Gaussian Denoiser: Residual Learning of Deep CNN for Image Denoising", 2017

A Quick Recap of Classical CNNs

A CNN is a Neural Network that uses a Convolutional Linear transformation before the non-linear activation function:

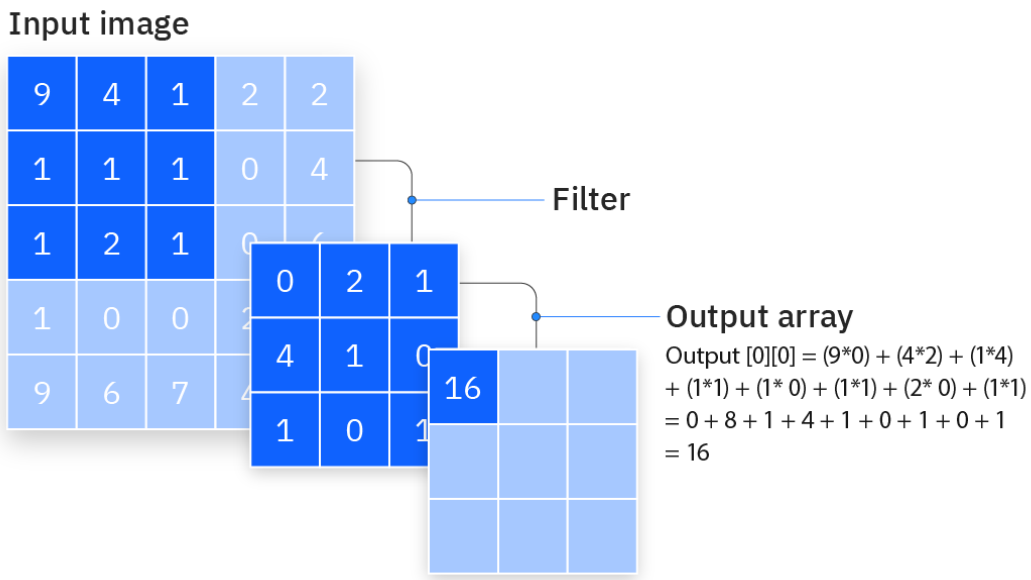


Figure: Visualization of Convolution

DnCNN: A Powerful Image Denoiser

Zhang et al. propose the following modifications to the CNN to obtain a simple and performany image denoising baseline they call DnCNN.

1. Residual Learning: Attempt to predict the noise v and recover $\hat{x} = y - \hat{v}$
2. Batch Normalization: Use BN after Convolution layers to stabilize training
3. Zero Padding: Zero-pad after every convolution to retain same image size

DnCNN Interpretation

- In the DnCNN framework, the model learns to strip the latent structure from the image, leaving only the noise.
- In general, Y is very similar to X, so trying to learn a mapping from Y to X is nearly an identity mapping, which deep neural networks struggle with, but v is very dissimilar from Y.
- The remaining Gaussian noise stabilizes BN, so Residual Learning and BN mutually benefit each other.
- Thus, DnCNN serves as a simple but powerful end-to-end baseline for Image Denoising.

Performance Metrics

To evaluate the regularizers, the authors focused on three key metrics:

- **Peak Signal-to-Noise Ratio (PSNR):**
  - Measured in decibels (dB).
  - The primary metric for image reconstruction quality.
  - Higher PSNR indicates less noise and closer proximity to ground truth.
- **Training Time:**
  - Measured in hours.
  - Captures the computational cost to learn the regularizer's parameters (spanning from 0.1 hours to over 41 hours).
- **Evaluation Time:**
  - Measured in seconds on a single NVIDIA RTX 4090 GPU (24GB).
  - Evaluates how fast the trained regularizer can solve the inverse problem for a given image.

Experimental Setup

The authors evaluated the regularizers across three distinct setups:

1. **Experiment 1 (Image Denoising):**
   - Models trained and tested on removing Gaussian noise ($\sigma = 0.1$) using the BSDS500/BSD68 datasets (standard image denoising datasets).
   - Establishes a baseline for expressivity and general image quality.
2. **Experiment 2 (Generalization to CT):**
   - Denoising models are tested "out-of-the-box" on the medical LoDoPaB-CT dataset without retraining.
   - Tests how well learned priors transfer to unseen inverse problems.
3. **Experiment 3 (CT-Specific Training):**
   - Models explicitly trained and evaluated on LoDoPaB-CT data.
   - Tests the performance ceiling when specialized for a complex domain.

Results: Convex & Weakly Convex Models

1. Convex Models (e.g., CRR, ICNN)

- **Context:** Highly constrained parameter spaces (CRR $\approx$ 15k, ICNN $\approx$ 26k).
- **Performance:** Yield the lowest PSNRs and overly smoothed textures among learned architectures, but consistently beat traditional Total Variation (TV).
- **Efficiency:** Unmatched speed. Training mostly takes under an hour, and evaluation is near-instant ($<$ 0.2 seconds).

2. Weakly Convex Models (e.g., WCRR)

- **Performance:** An effective middle ground. Relaxing strict convexity boosts PSNR over strictly convex models.
- **Efficiency:** WCRR aligns closely with CRR, retaining rapid training/evaluation speeds despite non-convexity.

Results: Convex & Weakly Convex Models

1. Convex Models (e.g., CRR, ICNN)

- **Context:** Highly constrained parameter spaces (CRR $\approx$ 15k, ICNN $\approx$ 26k).
- **Performance:** Yield the lowest PSNRs and overly smoothed textures among learned architectures, but consistently beat traditional Total Variation (TV).
- **Efficiency:** Unmatched speed. Training mostly takes under an hour, and evaluation is near-instant (< 0.2 seconds).

2. Weakly Convex Models (e.g., WCRR)

- **Performance:** An effective middle ground. Relaxing strict convexity boosts PSNR over strictly convex models.
- **Efficiency:** WCRR aligns closely with CRR, retaining rapid training/evaluation speeds despite non-convexity.

Results: Non-Convex Models

3. Non-Convex Models (e.g., IDCNN, TDV, LSR)

- **Context:** Much higher parameter counts (TDV $\approx$ 400k, LSR $\approx$ 4M).
- **Performance:** TDV and LSR achieve the highest PSNRs among variational methods, yielding crisp, well-defined details that rival deep supervised networks.
- **Training Sensitivity:** Highly fragile. They require expensive supervised "bilevel" training (BL-IFT/BL-JFB) taking up to 41 hours to reach top performance. Simpler schemes like Score Matching (SM) or Adversarial Regularization (AR) cause quality to collapse into cartoonish or overly smoothed images.
- **Evaluation:** Slower than convex models, taking 1–2 seconds per image.

Discussion & Main Takeaways

The Core Trade-Off: Guarantees vs. Expressivity

- **Convex Models:** Guarantee stable optimization and are remarkably cheap to train/evaluate, but heavily lack the parameters and capacity to reconstruct fine details.
- **Non-Convex Models:** Deliver high image quality, but are computationally expensive, have slower evaluation times, and are highly sensitive to their training methodology.

Visualizing Expressivity: Denoising Results



Figure: Comparison of denoising performance across regularization categories, highlighting the trade-off between mathematical structure and visual expressivity.

References I

- Amos, Brandon, Lei Xu, and J. Zico Kolter. "Input Convex Neural Networks". In: *Proceedings of the 34th International Conference on Machine Learning*. Ed. by Doina Precup and Yee Whye Teh. Vol. 70. Proceedings of Machine Learning Research. PMLR, 2017, pp. 146–155. URL: https://proceedings.mlr.press/v70/amos17b.html.
- Goujon, Alexis, Sebastian Neumayer, and Michael Unser. "Learning Weakly Convex Regularizers for Convergent Image-Reconstruction Algorithms". In: *SIAM Journal on Imaging Sciences* 17.1 (2024), pp. 91–115. DOI: 10.1137/23M1565243. eprint: https://doi.org/10.1137/23M1565243. URL: https://doi.org/10.1137/23M1565243.

References II

- Zhang, Kai et al. "Beyond a Gaussian Denoiser: Residual Learning of Deep CNN for Image Denoising". In: *Trans. Img. Proc.* 26.7 (July 2017), 3142–3155. ISSN: 1057-7149. DOI: 10.1109/TIP.2017.2662206. URL: https://doi.org/10.1109/TIP.2017.2662206.
- Zhang, Yasi and Oscar Leong. "Learning Difference-of-Convex Regularizers for Inverse Problems: A Flexible Framework with Theoretical Guarantees". In: (2025). arXiv: 2502.00240 [stat.ML]. URL: https://arxiv.org/abs/2502.00240.